

Engineering



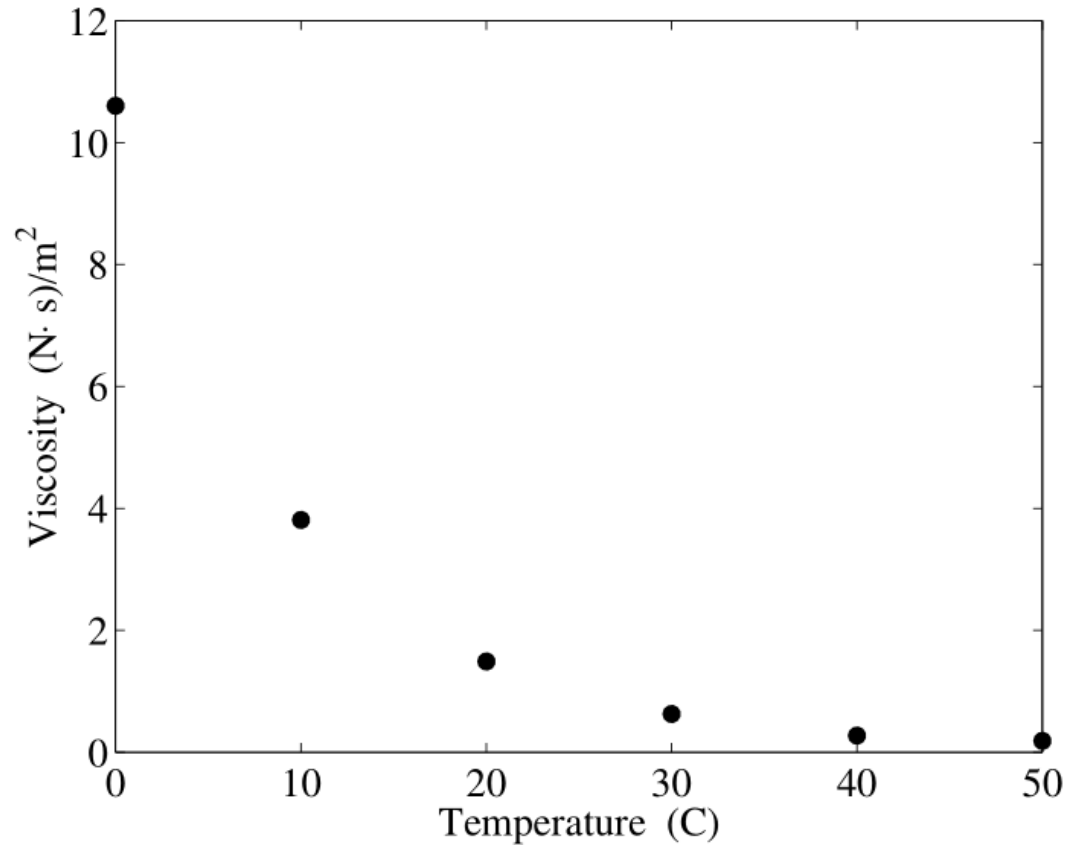
& Physics

PHYS 351

Derivatives

Dr. Daugherty
Abilene Christian University

Problem



Estimate the derivative of a function from data points.

Related: if I get to calculate my own data points, what is the optimal spacing to use?

Approaches

Strategies:

- 1) Fit the data to a known function, take the analytic derivative
- 2) Use cubic spline interpolation, take derivative of splines
- 3) Calculate the result “directly”



Errors

Two types of ultimately unavoidable errors in computing:

- 1) Round off – limited to around 16 sig figs (base 10)
- 2) Truncation – using finite approximation

Big O Notation

We can write truncation error with “Big O” notation

Suppose I know a function and all of its derivatives at x_0 .

Approximate the function at some step h away:

$$f(x_0 + h) \approx f(x_0) + hf'(x_0) + \frac{h^2}{2}f''(x_0) + \frac{h^3}{3!}f'''(x_0) + \dots$$

$$f(x_0 + h) \approx f(x_0) + O(h)$$

$$f(x_0 + h) \approx f(x_0) + hf'(x_0) + O(h^2)$$

Note that $h < 1$ (hopefully $h \ll 1$) so the leading term dominates and other terms are quite small

Derivation

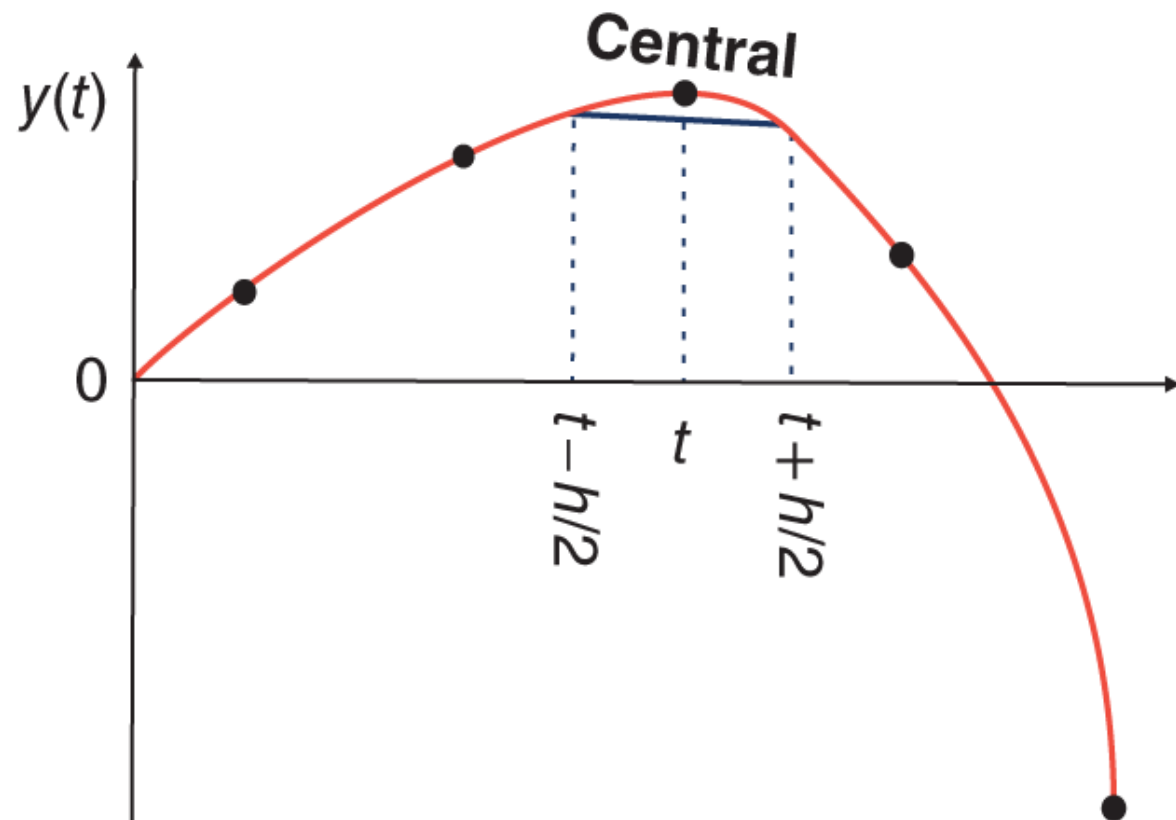
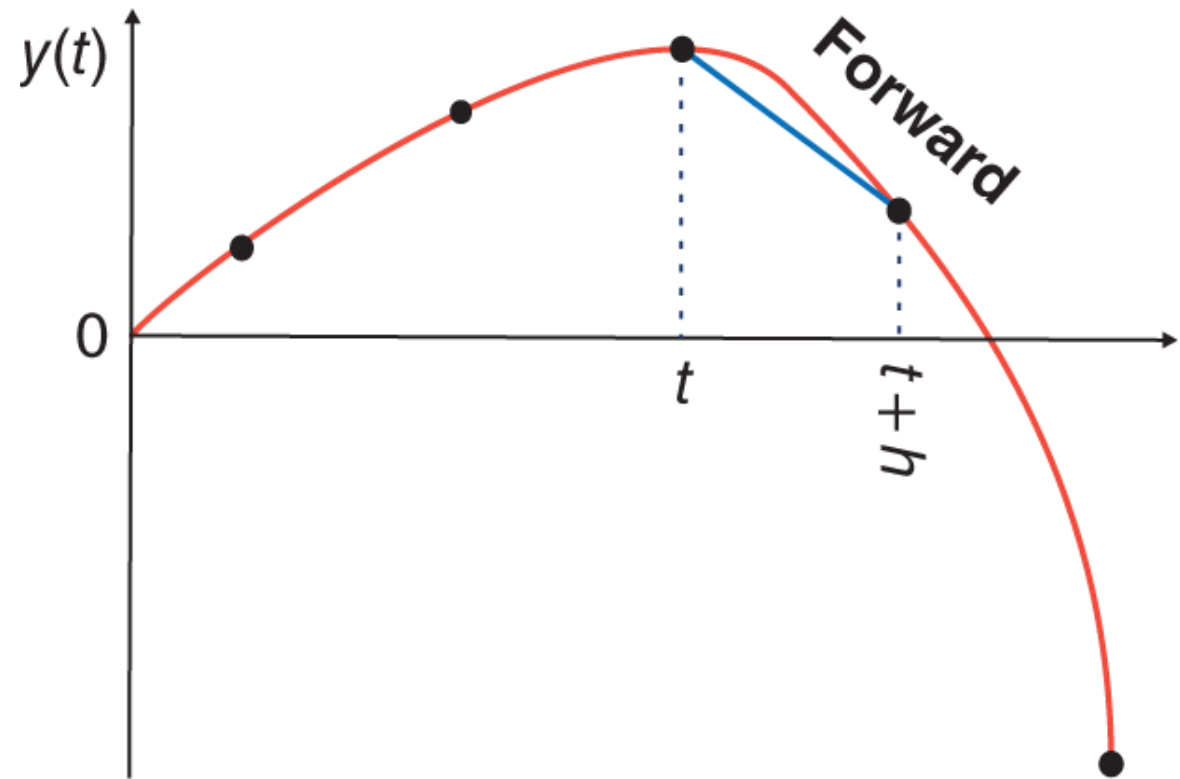
1) Forward Difference



Derivation

$$f(x_0 + h) \approx f(x_0) + hf'(x_0) + \frac{h^2}{2!}f''(x_0) + \frac{h^3}{3!}f'''(x_0) + \frac{h^4}{4!}f^{(4)}(x_0)$$

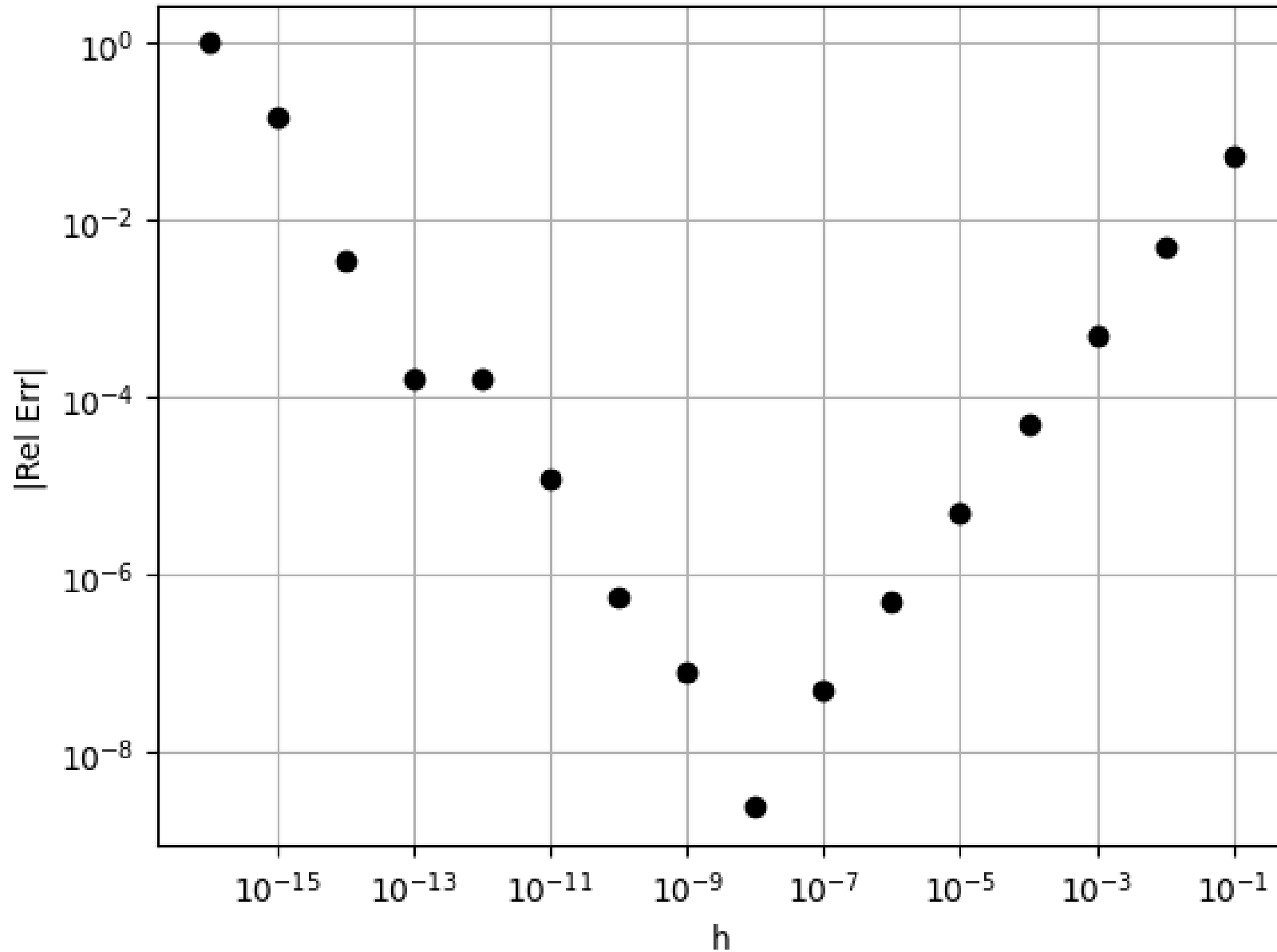
$$f(x_0 - h) \approx f(x_0) - hf'(x_0) + \frac{h^2}{2!}f''(x_0) - \frac{h^3}{3!}f'''(x_0) + \frac{h^4}{4!}f^{(4)}(x_0)$$



Challenge

Pick an easy-to-differentiate function $f(x)$ and an x_0 value to estimate $f'(x_0)$. What value of h gives you the smallest error?

Forward Diff Relative Error vs Step Size



$f(x) = \sin x$
at $x=1$

Using forward difference

Derivation

- 1) Forward Difference
- 2) Backward Difference
- 3) Centered Difference

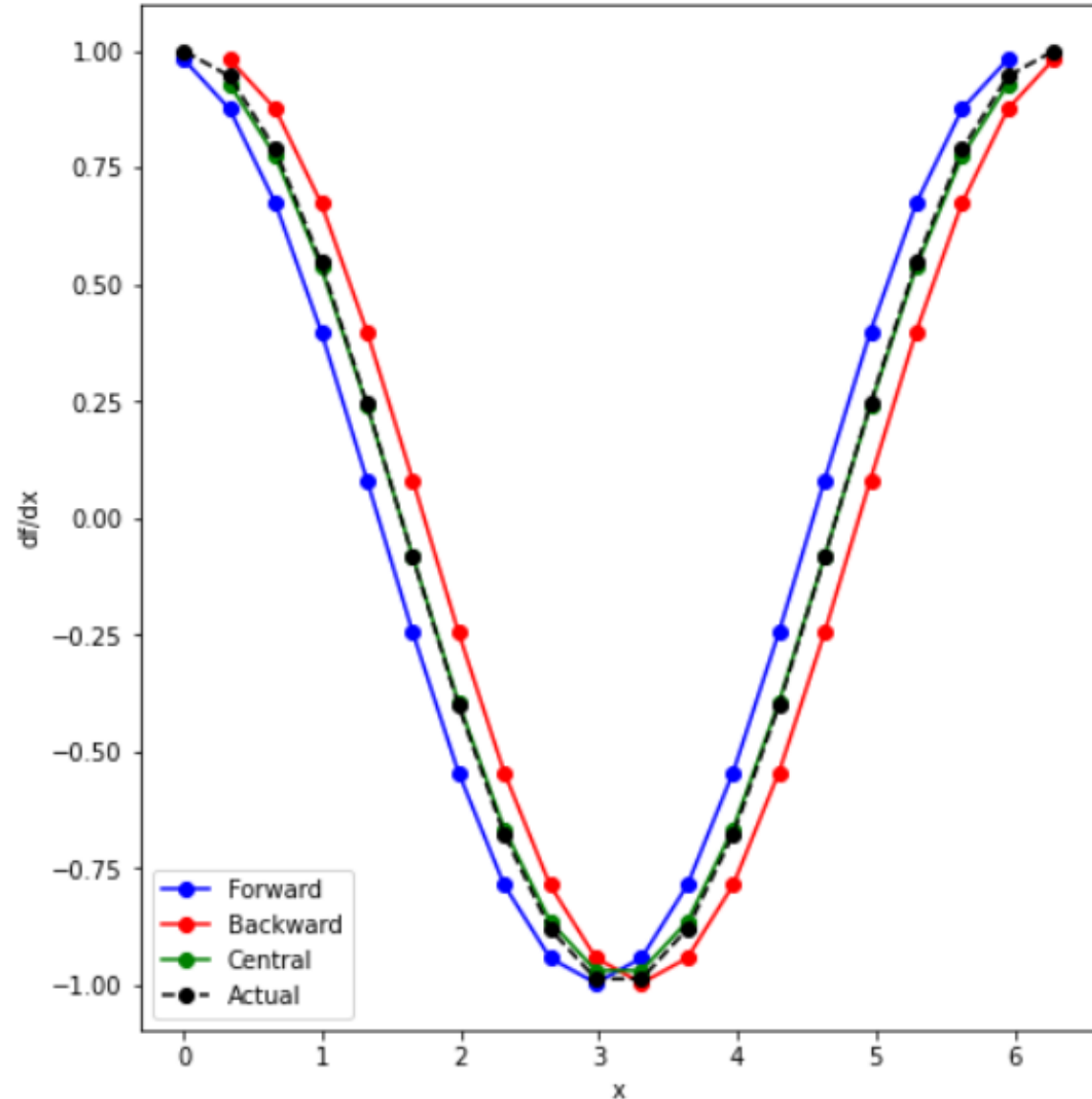


Challenge

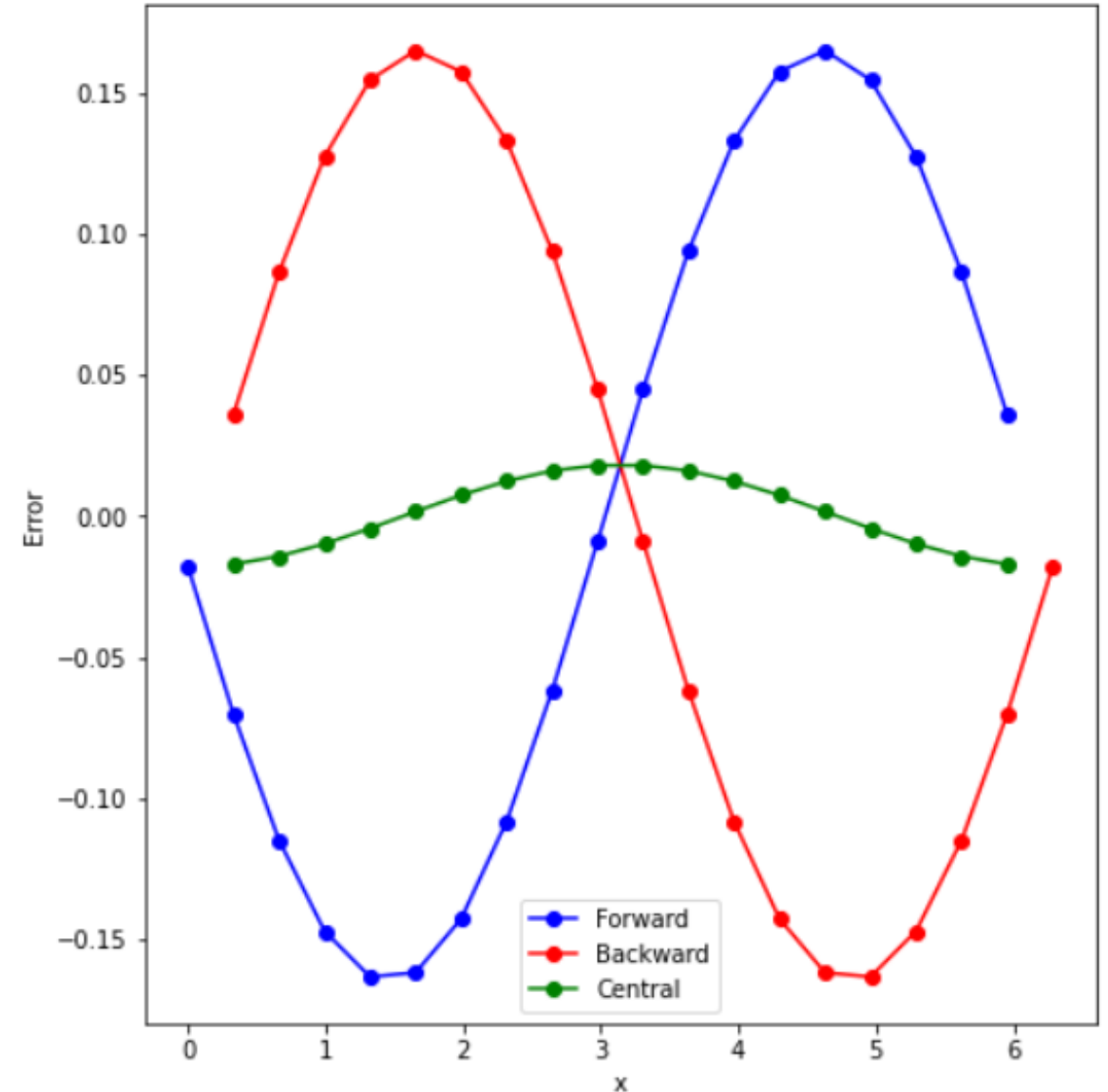
- 1) Check if centered difference really is more accurate
- 2) Now what value of h gives you the smallest error?

Taylor Series Methods

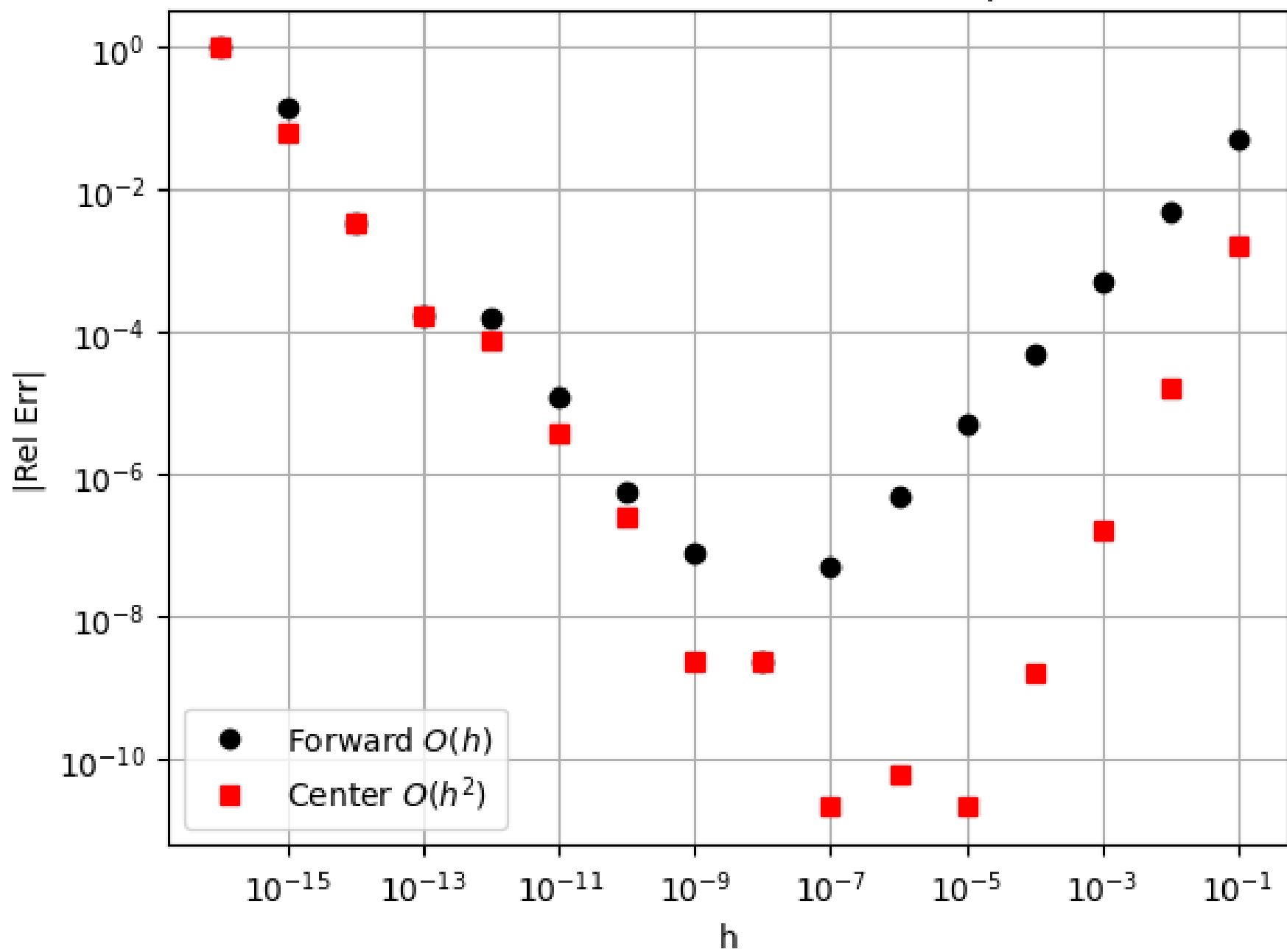
Derivatives



ERROR

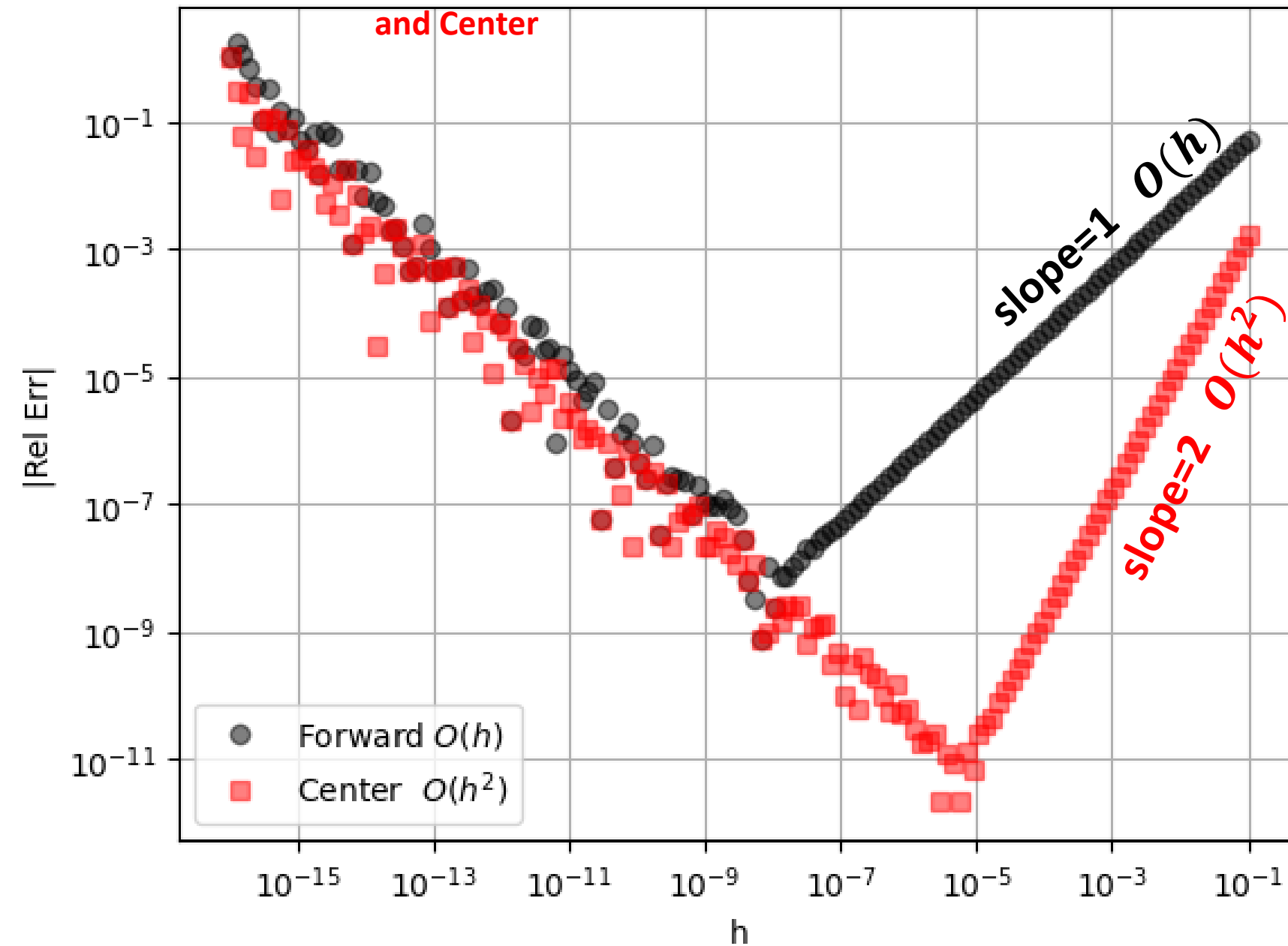


Forward Diff Relative Error vs Step Size



$f(x) = \sin x$
at $x=1$

Forward Diff Relative Error vs Step Size



Region on the left (small h)
the dominant error is
ROUNDING from subtractive
cancellation in the
numerator.

Region on the right (larger h)
the dominant error is
TRUNCATION from only
keeping lower order
approximations. The slopes
show the order of the error!

Richardson Extrapolation

Assume: $R = A(h) + E(h)$ where

- $A(h)$ is some approximation that depends on step size h
- $E(h)$ is the error with $E(h) \approx ch^p + \dots$

Combine to cancel leading error:

$$R = \frac{2^p A(h) - A(2h)}{2^p - 1}$$

Richardson Extrapolation

Forward difference: $A(h) = f'(x_0) \approx \frac{f(x_0+h)-f(x_0)}{h} + \frac{h}{2!} f''(x_0) + \frac{h^2}{3!} f'''(x_0)$

Use $p = 1$

$$R = \frac{2^p A(h) - A(2h)}{2^p - 1} = \frac{2A(h) - A(2h)}{1}$$

This should be an estimate for f' accurate to $O(h^2)$

We can do it the hard way to prove that it works:

$$2A(h) - A(2h) = \frac{2[f(x_0 + h) - f(x_0)]}{h} - \frac{[f(x_0 + 2h) - f(x_0)]}{2h} - \frac{h^2}{2} f'''(x_0)$$

$$2A(h) - A(2h) = \frac{-3f(x_0) + 4f(x_0 + h) - f(x_0 + 2h)}{2h} + O(h^2)$$

Continuous Function

<https://docs.scipy.org/doc/scipy-1.18.0/reference/generated/scipy.differentiate.derivative.html>

`scipy.differentiate.`

derivative

derivative(*f*, *x*, *, *args=()*, *kwargs=None*, *tolerances=None*, *maxiter=10*, *order=8*, *initial_step=0.5*, *step_factor=2.0*, *step_direction=0*, *preserve_shape=False*, *callback=None*) [\[source\]](#)

Evaluate the derivative of an elementwise, real scalar function numerically.

For each element of the output of *f*, `derivative` approximates the first derivative of *f* at the corresponding element of *x* using finite difference differentiation.

This function works elementwise when *x*, *step_direction*, and *args* contain (broadcastable) arrays.

Parameters:

f : callable

The function whose derivative is desired. The signature must be:

```
f(xi: ndarray, *argsi) → ndarray
```

where each element of `xi` is a finite real number and `argsi` is a tuple, which may contain an arbitrary number of arrays that are broadcastable with `xi`. *f* must be an elementwise function: each scalar element `f(xi)[j]` must equal `f(xi[j])` for valid indices `j`. It must not mutate the array `xi` or the arrays in `argsi`.

x : float array_like

Abscissae at which to evaluate the derivative. Must be broadcastable with *args* and *step_direction*.

```
>>> import numpy as np
>>> from scipy.differentiate import derivative
>>> f = np.exp
>>> df = np.exp # true derivative
>>> x = np.linspace(1, 2, 5)
>>> res = derivative(f, x)
>>> res.df # approximation of the derivative
array([2.71828183, 3.49034296, 4.48168907, 5.75460268, 7.3890561 ])
>>> res.error # estimate of the error
array([7.13740178e-12, 9.16600129e-12, 1.17594823e-11, 1.51061386e-11,
       1.94262384e-11])
>>> abs(res.df - df(x)) # true error
array([2.53130850e-14, 3.55271368e-14, 5.77315973e-14, 5.59552404e-14,
       6.92779167e-14])
```

Data Points

Easy! Use `numpy.gradient`. Lots of choices for 2nd parameter to specify dx

`numpy.gradient(f, *varargs, axis=None, edge_order=1)`

[\[source\]](#)

Return the gradient of an N-dimensional array.

The gradient is computed using second order accurate central differences in the interior points and either first or second order accurate one-sides (forward or backwards) differences at the boundaries. The returned gradient hence has the same shape as the input array.

Parameters: **f** : *array_like*

An N-dimensional array containing samples of a scalar function.

varargs : *list of scalar or array, optional*

Spacing between f values. Default unitary spacing for all dimensions. Spacing can be specified using:

1. single scalar to specify a sample distance for all dimensions.
2. N scalars to specify a constant sample distance for each dimension. i.e. dx , dy , dz , ...
3. N arrays to specify the coordinates of the values along each dimension of F.
The length of the array must match the size of the corresponding dimension
4. Any combination of N scalars/arrays with the meaning of 2. and 3.

Higher Order

https://en.wikipedia.org/wiki/Finite_difference_coefficient

Derivative	Accuracy	-5	-4	-3	-2	-1	0	1	2	3	4	5
1	2					-1/2	0	1/2				
	4				1/12	-2/3	0	2/3	-1/12			
	6			-1/60	3/20	-3/4	0	3/4	-3/20	1/60		
	8		1/280	-4/105	1/5	-4/5	0	4/5	-1/5	4/105	-1/280	
2	2					1	-2	1				
	4				-1/12	4/3	-5/2	4/3	-1/12			
	6			1/90	-3/20	3/2	-49/18	3/2	-3/20	1/90		
	8		-1/560	8/315	-1/5	8/5	-205/72	8/5	-1/5	8/315	-1/560	
3	2				-1/2	1	0	-1	1/2			
	4			1/8	-1	13/8	0	-13/8	1	-1/8		
	6		-7/240	3/10	-169/120	61/30	0	-61/30	169/120	-3/10	7/240	
4	2				1	-4	6	-4	1			
	4			-1/6	2	-13/2	28/3	-13/2	2	-1/6		
	6		7/240	-2/5	169/60	-122/15	91/8	-122/15	169/60	-2/5	7/240	
5	2			-1/2	2	-5/2	0	5/2	-2	1/2		
	4		1/6	-3/2	13/3	-29/6	0	29/6	-13/3	3/2	-1/6	
	6	-13/288	19/36	-87/32	13/2	-323/48	0	323/48	-13/2	87/32	-19/36	13/288

For example, the third derivative with a second-order accuracy is

$$f'''(x_0) \approx \frac{-\frac{1}{2}f(x_{-2}) + f(x_{-1}) - f(x_{+1}) + \frac{1}{2}f(x_{+2})}{h_x^3} + O(h_x^2)$$

<https://web.media.mit.edu/~crtaylor/calculator.html> - nice tool which gives python code!

Higher Order

https://en.wikipedia.org/wiki/Finite_difference_coefficient

Also works for forwards and backwards

Derivative	Accuracy	0	1	2	3	4	5	6	7
1	1	-1	1						
	2	-3/2	2	-1/2					
	3	-11/6	3	-3/2	1/3				
	4	-25/12	4	-3	4/3	-1/4			
	5	-137/60	5	-5	10/3	-5/4	1/5		
	6	-49/20	6	-15/2	20/3	-15/4	6/5	-1/6	
2	1	1	-2	1					
	2	2	-5	4	-1				
	3	35/12	-26/3	19/2	-14/3	11/12			
	4	15/4	-77/6	107/6	-13	61/12	-5/6		
	5	203/45	-87/5	117/4	-254/9	33/2	-27/5	137/180	
	6	469/90	-223/10	879/20	-949/18	41	-201/10	1019/180	-7/10

Cubic Spline Interpolation

[https://github.com/mdaughterity/Numerical2024/blob/main/fits/Week 6 Interpolation.ipynb](https://github.com/mdaughterity/Numerical2024/blob/main/fits/Week%206%20Interpolation.ipynb)

```
# Make fake data for object in free-fall
```

```
tdata = np.arange(0,6)
```

```
v0 = 20
```

```
g = 9.8
```

```
ydata = v0*tdata - 0.5*g*tdata**2
```

```
# Fit cubic spline
```

```
yspline = CubicSpline(tdata, ydata)
```

```
vspline = yspline.derivative() # automatically get derivative
```

```
ts = np.linspace(0,5)
```

```
plt.figure(figsize=(10,4))
```

```
plt.subplot(1,2,1)
```

```
plt.plot(tdata, ydata, 'ko')
```

```
plt.plot(ts,yspline(ts))
```

```
plt.title('Height')
```

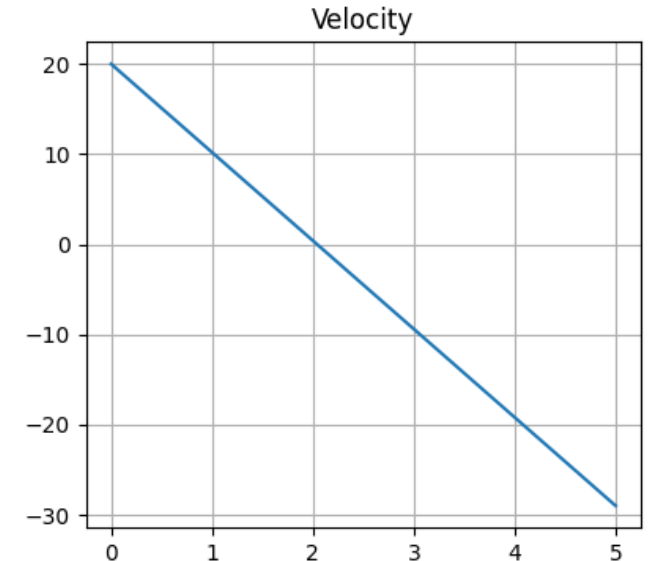
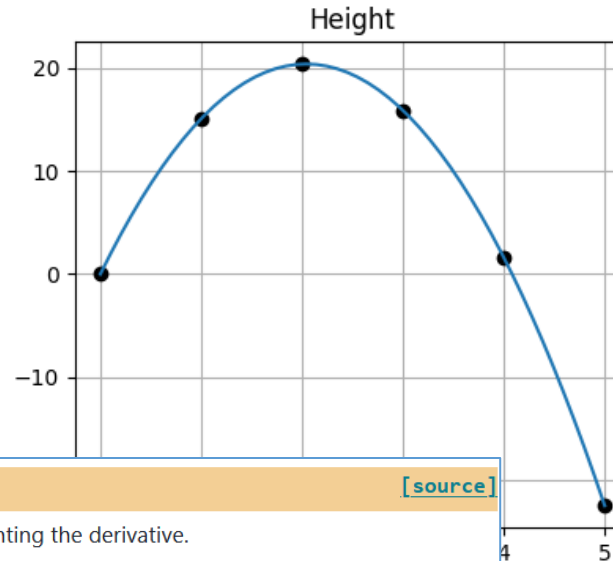
```
plt.grid()
```

```
plt.subplot(1,2,2)
```

```
plt.plot(ts,vspline(ts))
```

```
plt.title('Velocity')
```

```
plt.grid()
```



derivative(*nu=1*)

[\[source\]](#)

Construct a new piecewise polynomial representing the derivative.

Parameters:

nu : *int, optional*

Order of derivative to evaluate. Default is 1, i.e., compute the first derivative. If negative, the antiderivative is returned.

Returns:

pp : *PPoly*

Piecewise polynomial of order $k_2 = k - n$ representing the derivative of this polynomial.

Notes

Derivatives are evaluated piecewise for each polynomial segment, even if the polynomial is not differentiable at the breakpoints. The polynomial intervals are considered half-open, $[a, b)$, except for the last interval which is closed $[a, b]$.

<https://docs.scipy.org/doc/scipy/reference/generated/scipy.interpolate.CubicSpline.derivative.html#scipy.interpolate.CubicSpline.derivative>

DrD's Advice

- Data points (that you can't change):
 - use `numpy.gradient` for centered difference or spline interpolation
 - Noisy data? You should fit it first and differentiate the fit
- Function (that you can sample):
 - use `scipy.derivative`
 - sample it with `linspace` and use `numpy.gradient` (remember that error goes as h^2)
- If I really had a situation where this wasn't good enough I would just use another library instead of looking up and implementing higher-order stuff myself:
 - <https://github.com/maroba/findiff>
 - <https://github.com/pbrod/numdifftools>

Things to Know

- Roundoff vs Truncation Errors
- CENTERED DIFFERENCE IS BEST
- Use `numpy.gradient` for data points
- Use `scipy.diffentiate.derivative` for functions
- Fitting / Spline Interpolation is also an option

most accurate step
size is about $h=1e-5$